



Spring AOP

Ce que vous avez déjà acquis

Ces bases constituent le socle pour maîtriser l'AOP Spring.

Java EE

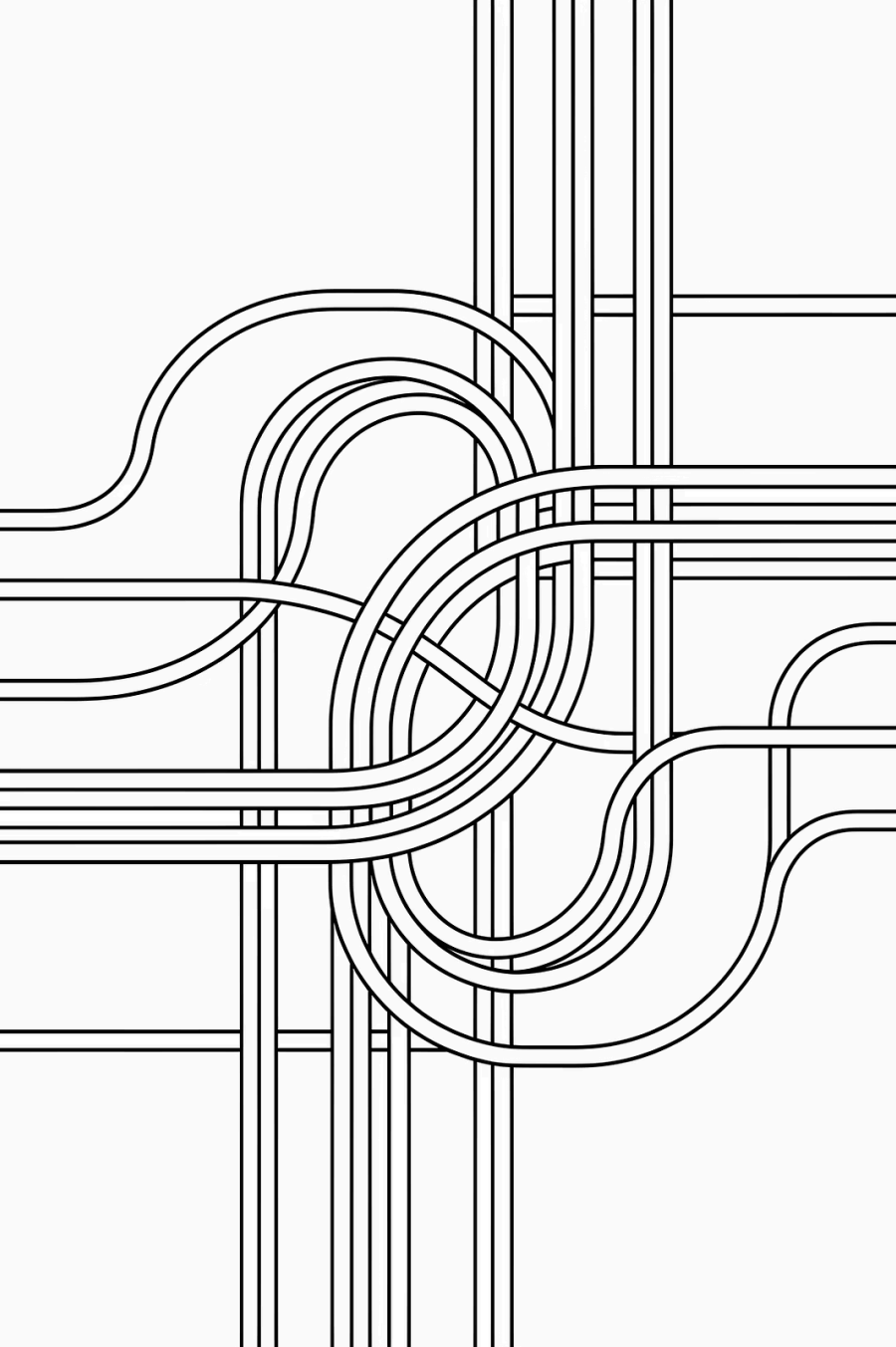
Servlets, JSP, architecture entreprise

Spring & Spring Boot

Écosystème Spring, Initializr, auto-configuration

Configuration & DI

IoC Container, @Component, @Autowired



Le problème des préoccupations transverses

Dans toute application d'entreprise, certaines fonctionnalités traversent horizontalement toutes les couches : journalisation, sécurité, transactions, gestion des erreurs. On parle de **cross-cutting concerns**.

Avant l'AOP, ces comportements étaient dupliqués dans chaque classe, chaque méthode — aboutissant à un code difficile à maintenir.

Sans AOP vs. Avec AOP

Sans AOP

- Logique de journalisation copiée dans chaque service
- Gestion des transactions répétée partout
- Code métier pollué par des préoccupations techniques
- Violation du principe DRY

Avec AOP

- Préoccupations transverses isolées dans des **aspects**
- Code métier pur et lisible
- Application centralisée et cohérente des comportements
- Respect du principe SRP

Concepts fondamentaux de l'AOP

Aspect

Module encapsulant une préoccupation transverse — l'équivalent d'une classe pour les comportements transversaux.

Join Point

Point d'exécution où un aspect peut s'appliquer : appel de méthode, levée d'exception, accès à un champ.

Advice

Action exécutée par un aspect à un join point. Définit *quand et quoi* exécuter.

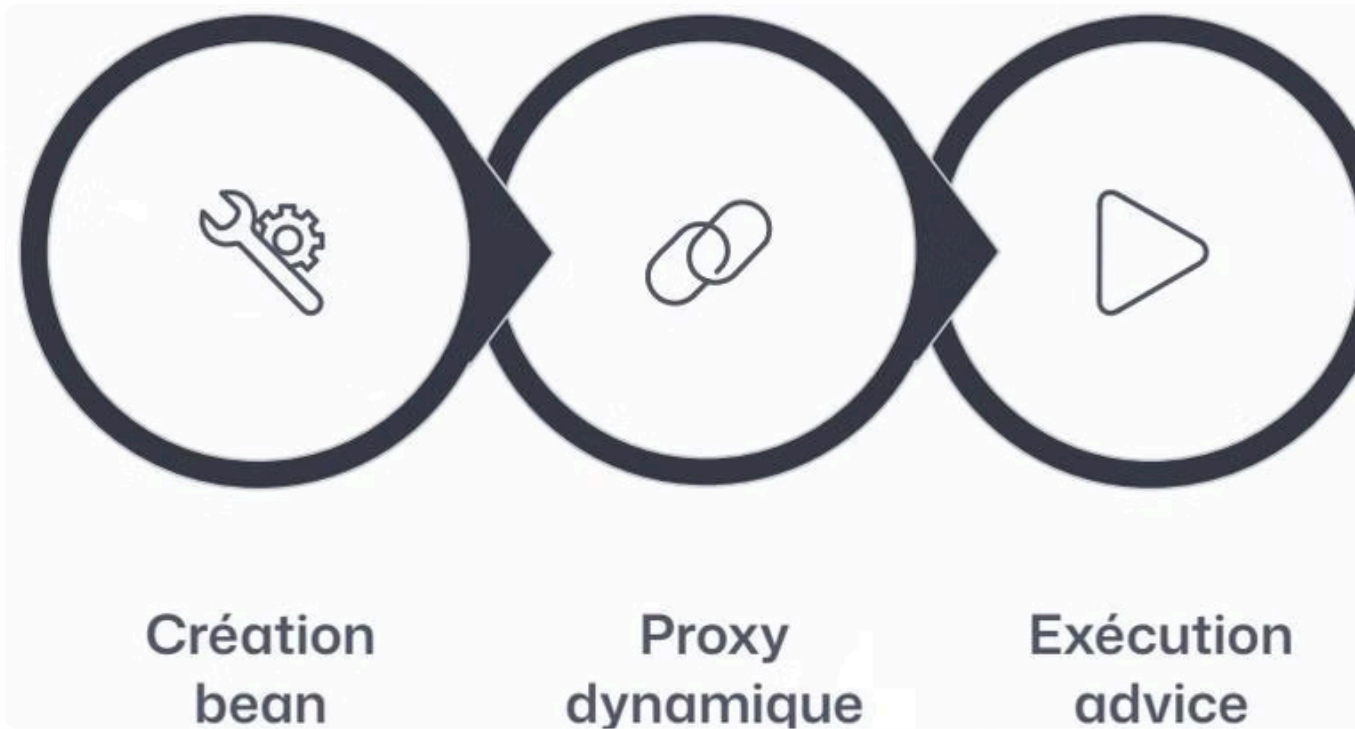
Pointcut

Expression déclarative qui sélectionne un ensemble de join points. Définit *où* s'applique l'aspect.

Weaving

Assemblage des aspects avec les objets de l'application. Spring effectue ce tissage à l'exécution via des proxies.

Le mécanisme de proxy dans Spring AOP



Lorsqu'un bean est annoté `@Aspect`, Spring crée automatiquement un **proxy** autour de cet objet. Chaque appel de méthode passe par ce proxy, qui détermine si des advices doivent s'exécuter.

- ❗ Spring AOP ne supporte que les join points de type **appel de méthode** sur des beans Spring. Pour un tissage plus fin (champs, constructeurs), il faut utiliser **AspectJ** complet.

Les 5 types d'Advices



@Before

Exécuté avant la méthode cible.



@After

Après la méthode, succès ou exception — équivalent d'un `finally`.



@AfterReturning

Après un retour normal. Peut accéder à la valeur de retour.



@AfterThrowing

Après qu'une exception est levée. Centralise la gestion des erreurs.



@Around

Le plus puissant : enveloppe complètement la méthode. Peut modifier arguments, retour et exceptions.

Pointcuts – Syntaxe et usage

execution() – Le plus courant

Cible des méthodes selon leur signature : `execution(* com.example.service.*(..))`

within() – Par type

Tous les join points dans un package :
`within(com.example.service..*)`

@annotation() – Par annotation

Méthodes portant une annotation spécifique :
`@annotation(com.example.Loggable)`

bean() – Par nom de bean

Cible un bean Spring spécifique : `bean(userService)`

- ❏ Les opérateurs `&&`, `||` et `!` permettent de combiner plusieurs pointcuts. Nommez et réutilisez vos pointcuts avec `@Pointcut` sur une méthode vide.

Anatomie d'une expression execution()

```
execution([modificateur] type-retour [classe.]nom-méthode(paramètres))
```

```
// Toutes les méthodes publiques
```

```
execution(public * *(..))
```

```
// Toutes les méthodes de UserService
```

```
execution(* com.example.service.UserService.*(..))
```

```
// Méthodes "save*", premier param String, dans tout *Service
```

```
execution(* com.example.*Service.save*(String,..))
```

```
// Méthodes publiques annotées @Transactional
```

```
execution(public @Transactional * *(..))
```

Déclarer un Aspect avec Spring

```
@Aspect
@Component
public class LoggingAspect {

    private static final Logger log =
        LoggerFactory.getLogger(LoggingAspect.class);

    @Pointcut("execution(* com.example.service.*(..))")
    public void serviceLayer() {}

    @Before("serviceLayer()")
    public void logBefore(JoinPoint joinPoint) {
        log.info("Appel méthode : {}",
            joinPoint.getSignature().getName());
    }

    @AfterReturning(pointcut = "serviceLayer()", returning = "result")
    public void logAfterReturning(JoinPoint jp, Object result) {
        log.info("Méthode {} retourne : {}",
            jp.getSignature().getName(), result);
    }
}
```

Aspects pour la journalisation

La journalisation est l'exemple canonique de préoccupation transverse. Un aspect bien conçu enregistre entrées, sorties et exceptions **sans encombrer chaque méthode de code technique**.

DEBUG

Entrées et sorties de méthodes

INFO

Événements métier importants

WARN

Situations anormales mais gérées

ERROR

Erreurs critiques



Aspect de journalisation des performances

```
@Aspect
@Component
public class PerformanceLoggingAspect {

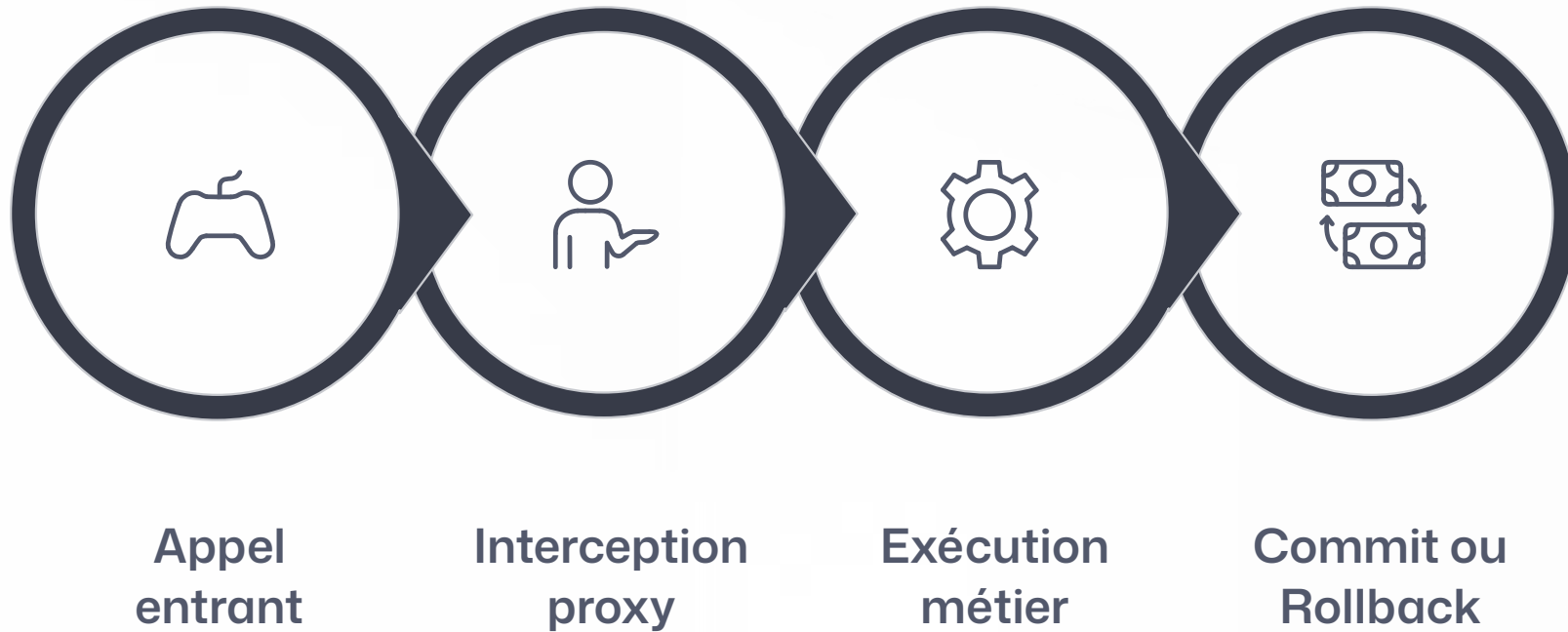
    @Around("execution(* com.example.service.*.*(..))")
    public Object profileServiceMethods(ProceedingJoinPoint pjp)
        throws Throwable {

        String methodName = pjp.getSignature().toShortString();
        log.debug("→ Entrée [{}] args={}", methodName,
            Arrays.toString(pjp.getArgs()));

        long startTime = System.currentTimeMillis();
        try {
            Object result = pjp.proceed();
            long duration = System.currentTimeMillis() - startTime;
            log.debug("← Sortie [{}] en {}ms", methodName, duration);
            return result;
        } catch (Exception ex) {
            log.error("Exception [{}] : {}", methodName, ex.getMessage());
            throw ex;
        }
    }
}
```

- ✔ L'advice `@Around` avec `ProceedingJoinPoint` peut mesurer la durée, modifier les arguments ou le résultat, et décider de ne pas appeler la méthode cible.

Gestion des transactions avec l'AOP



Spring implémente `@Transactional` précisément grâce à l'AOP. Comprendre ce mécanisme permet d'éviter les pièges courants, notamment la **self-invocation** qui court-circuite le proxy.

@Transactional – Paramètres clés

propagation

`REQUIRED` (défaut) : rejoint ou crée une transaction.

`REQUIRES_NEW` : toujours une nouvelle transaction.

isolation

Niveau d'isolation vis-à-vis des transactions concurrentes :

`READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`.

rollbackFor

Par défaut, rollback uniquement sur `RuntimeException`.

Utilisez `rollbackFor = Exception.class` pour les checked exceptions.

readOnly / timeout

`readOnly = true` optimise les lectures. `timeout` annule automatiquement une transaction trop longue.

Pièges courants avec @Transactional

⚠ Self-invocation

Appel de `this.autreMethode()` contourne le proxy AOP. Solution : injecter le bean lui-même ou refactoriser dans un autre service.

⚠ Visibilité de la méthode

Spring AOP ne proxifie que les méthodes `public`. Une méthode `private` annotée `@Transactional` est silencieusement ignorée.

⚠ Checked exceptions non rollbackées

Par défaut, seules les `RuntimeException` déclenchent un rollback. Configurer explicitement `rollbackFor` pour les autres.

Annotations personnalisées pour les aspects

Créer ses propres annotations métier est l'un des patterns les plus élégants de l'AOP Spring : le code devient déclaratif, auto-documenté, et le ciblage est explicite.

```
// 1. Définir l'annotation
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Auditable {
    String action() default "";
    AuditLevel level() default AuditLevel.INFO;
}

// 2. Utilisation dans le code métier
@Service
public class OrderService {

    @Auditable(action = "CREATE_ORDER", level = AuditLevel.INFO)
    public Order createOrder(CreateOrderRequest request) {
        // Logique métier pure, sans code d'audit
        return orderRepository.save(new Order(request));
    }
}
```

Aspect d'audit avec @Auditable

@Aspect

@Component

```
public class AuditAspect {
```

@Autowired

```
private AuditService auditService;
```

@AfterReturning(

```
pointcut = "@annotation(auditable)",
```

```
returning = "result")
```

```
public void auditAction(
```

```
JoinPoint jp,
```

```
Auditable auditable,
```

```
Object result) {
```

```
    auditService.log(
```

```
        auditable.action(),
```

```
        auditable.level(),
```

```
        jp.getArgs(),
```

```
        result,
```

```
        SecurityContextHolder.getContext()
```

```
            .getAuthentication().getName()
```

```
    );
```

```
}
```

```
}
```

Aspect de sécurité personnalisé

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequiresRole {
    String[] value();
}

@Aspect
@Component
public class SecurityAspect {

    @Before("@annotation(requiresRole)")
    public void checkPermission(JoinPoint jp, RequiresRole requiresRole) {
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();
        if (auth == null || !auth.isAuthenticated())
            throw new UnauthorizedException("Non authentifié");

        boolean hasRole = Arrays.stream(requiresRole.value())
            .anyMatch(role -> auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_" + role)));
        if (!hasRole)
            throw new ForbiddenException("Rôles requis : " +
                Arrays.toString(requiresRole.value()));
    }
}

// Utilisation
@RequiresRole({"ADMIN", "MANAGER"})
public void deleteUser(Long userId) { ... }
```

 Cet exemple est pédagogique. En production, préférez Spring Security qui gère de nombreux cas supplémentaires.

Ordonner les aspects avec @Order

Lorsque plusieurs aspects s'appliquent au même join point, `@Order` contrôle leur séquence. Un ordre plus petit = priorité plus haute.



@Order(1) – Sécurité

Vérification des permissions en premier. Si l'accès est refusé, les autres aspects ne s'exécutent pas.



@Order(2) – Journalisation

Enregistrement de l'appel après validation de la sécurité. Capture les paramètres et le résultat.



@Order(3) – Transactions

Gestion transactionnelle au plus près de la méthode métier pour minimiser la durée de la transaction ouverte.

Expressions pointcut avancées

Composition de pointcuts

Utilisez `&&` pour combiner :

`execution(* *(..)) &&`

`@annotation(Loggable)`. Restreint
finement le ciblage.

Liaison de paramètres – `args()`

`args(userId, ..)` capture le premier
argument et le rend disponible dans
l'advice.

`target()` et `this()`

`target()` cible l'objet invoqué, `this()`
cible le proxy AOP lui-même. Utiles
pour restreindre à un type spécifique.

 Définissez vos pointcuts dans une classe dédiée `PointcutDefinitions` et référencez-les dans vos aspects pour éviter la duplication.

Configuration de Spring AOP

Dépendance Maven

```
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-aop
  </artifactId>
</dependency>
```

Configuration Java

```
@Configuration
@EnableAspectJAutoProxy(
    proxyTargetClass = true
)
public class AopConfig {
    // proxyTargetClass=true
    // force l'usage de CGLIB
    // même pour les interfaces
}
```

`proxyTargetClass = true` force les proxies CGLIB basés sur sous-classe, évitant certains problèmes d'injection.

Bonnes pratiques de l'AOP Spring

Cibler avec précision

Des pointcuts trop larges instrumentent des beans du framework.
Toujours préfixer par le package applicatif.

Un aspect = une responsabilité

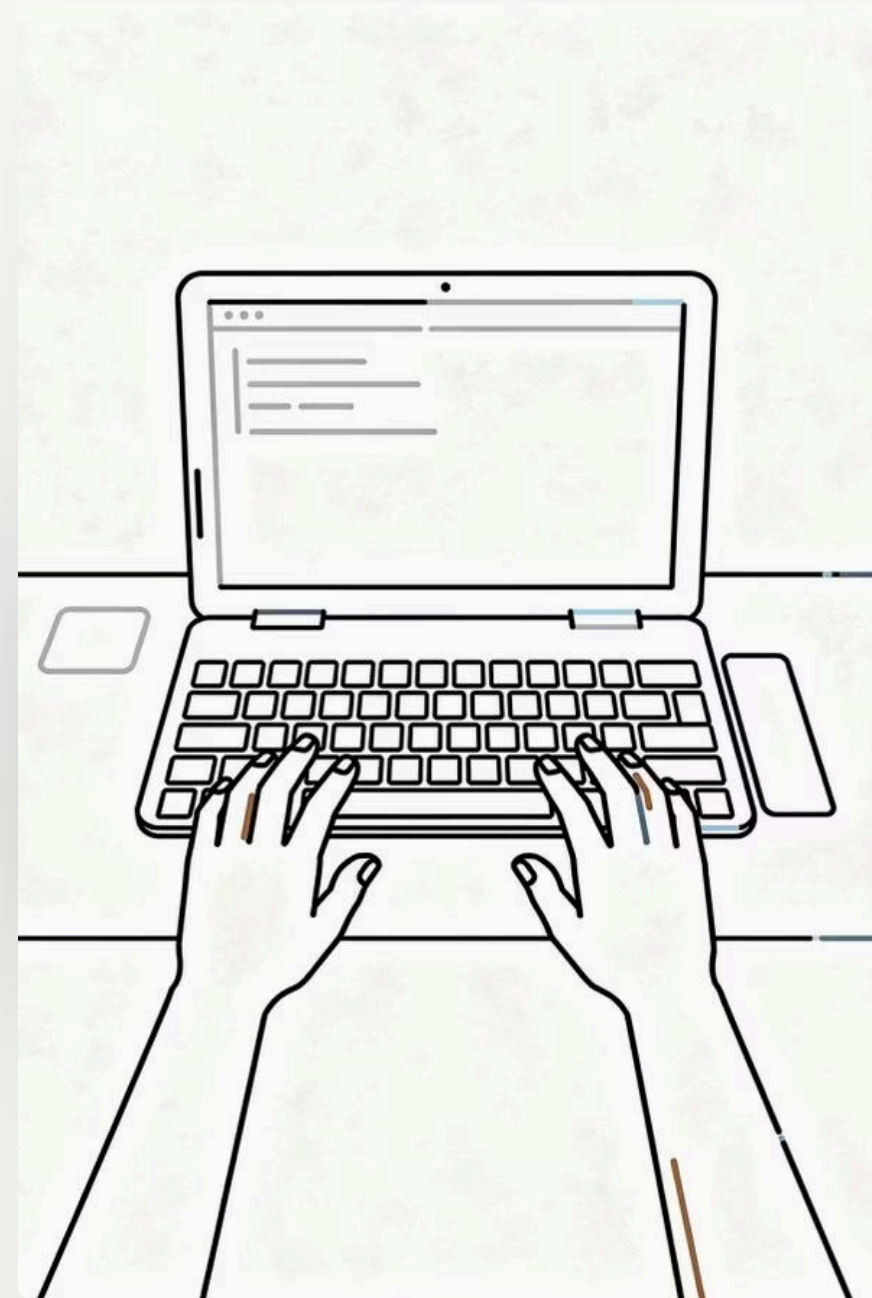
Ne regroupez pas journalisation, audit et sécurité dans le même aspect.
Chaque aspect doit avoir une responsabilité unique.

Documenter les aspects

Les aspects sont invisibles à la lecture du code. Documentez leur existence, portée et comportement dans le README.

Profiler en développement

Activez les logs AOP (niveau DEBUG) pour vérifier quels beans sont proxifiés. Mesurez l'impact sur les temps de réponse.





Tester les aspects AOP

Les aspects ne s'activent que dans un contexte Spring avec proxy — ils ne peuvent pas être testés en instanciant simplement la classe.

Test d'intégration

Utilisez `@SpringBootTest` avec le bean proxifié par Spring. Capturez les logs via `ListAppender` pour valider que l'aspect s'applique.

Test unitaire de la logique

Testez la logique interne de l'aspect sans Spring en mockant le `ProceedingJoinPoint` avec Mockito.

Tester les aspects – Exemple de code

```
@SpringBootTest
class LoggingAspectIntegrationTest {

    @Autowired
    private UserService userService; // Bean proxifié

    @MockBean
    private UserRepository userRepository;

    @Test
    void shouldLogMethodInvocation() {
        ListAppender<ILoggingEvent> listAppender =
            setupLogCapture(LoggingAspect.class);

        when(userRepository.findById(1L))
            .thenReturn(Optional.of(new User()));

        userService.findById(1L); // Déclenche le proxy AOP

        assertThat(listAppender.list)
            .extracting(ILoggingEvent::getMessage)
            .anyMatch(msg -> msg.contains("findById"));
    }
}
```

Récapitulatif

Ce que vous maîtrisez maintenant

Concepts AOP

Aspect, Join Point, Advice, Pointcut, Weaving — le vocabulaire et les mécanismes fondamentaux.

Proxies Spring

JDK vs CGLIB, self-invocation, limites du proxy — éviter les pièges courants.

Advices & Pointcuts

Les 5 types d'advices, syntaxe `execution()`, composition d'expressions.

Aspects métier

Journalisation, audit, sécurité, transactions — avec annotations personnalisées.

Glossaire AOP

Aspect	Module AOP encapsulant une préoccupation transverse (journalisation, sécurité, transactions)
Advice	Code exécuté par un aspect à un join point (Before, After, Around, AfterReturning, AfterThrowing)
Join Point	Point d'exécution où un aspect peut intervenir. Dans Spring AOP : uniquement les appels de méthodes sur des beans
Pointcut	Expression déclarative qui sélectionne un ensemble de join points
Weaving	Assemblage des aspects avec le code cible. Spring AOP utilise le weaving à l'exécution via des proxies
Proxy AOP	Objet intermédiaire créé par Spring pour envelopper un bean et intercepter les appels de méthodes
Self-invocation	Appel interne d'une méthode du même bean, contournant le proxy AOP et annulant les effets des aspects
Cross-cutting concern	Préoccupation transverse qui s'applique à de nombreux endroits de l'application

Quiz de consolidation

1

@After vs @AfterReturning

Quelle est la différence ? Dans quel cas précis utiliseriez-vous l'un plutôt que l'autre ?

2

Lecture de pointcut

Que cible `execution(* com.example..service.*Service.find*(..))` ? Comment exclure les méthodes `findAll` ?

3

@Transactional sur méthode privée

Pourquoi l'annotation est-elle silencieusement ignorée ? Comment contourner ce problème ?

Quiz – Questions avancées

1

@Order et aspects multiples

Trois aspects s'appliquent au même join point. Comment garantir que la vérification de sécurité s'exécute toujours en premier ?

2

Communication découplée

Comment implémenter l'envoi d'un email après la création d'un utilisateur, sans que `UserService` ne connaisse `EmailService` ?

3

Annotation personnalisée

Concevez une annotation `@RateLimit` et l'aspect correspondant pour limiter le nombre d'appels par minute sur une méthode.